TeamBlog 实施方案 v1.2 · 三服务架构

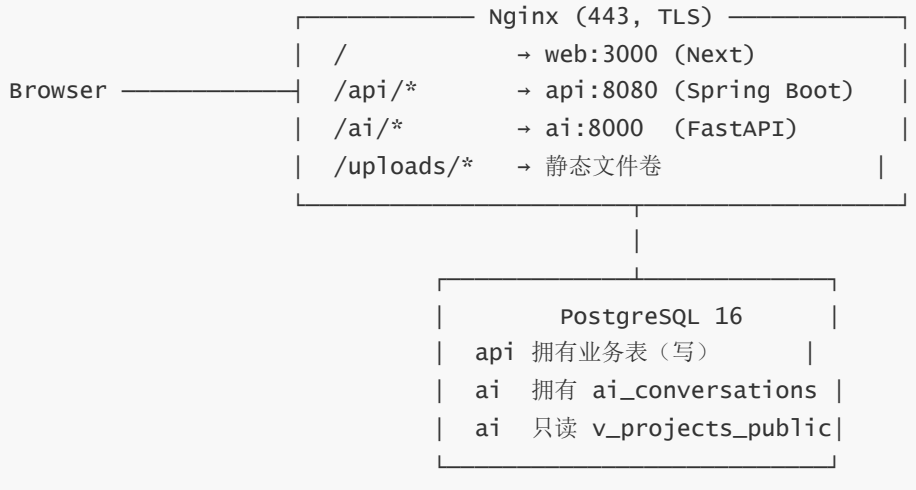
> **本文是变更文档，只写相对 v1.1 的差异。** 未提及的章节（§3 数据模型字段、§4.4 边界规则、§4.5 单测用例、§7.1/7.2 prompt、§13 风险）继续以 v1.1 为准。
>
> 变更原因：后端改用 Spring Boot、AI 改用 FastAPI，v1.1 中"Next.js 全栈同仓 + Prisma + 自建 Node 认证"的假设全部失效，需要重新划分服务边界、认证位置、迁移工具与部署拓扑。

0. 变更摘要

维度	v1.1（Next 全栈）	v1.2（三服务）	影响
运行时	Next.js 一个进程	Next（UI）+ Spring Boot（API）+ FastAPI（AI）	3 个镜像、3 套构建、4 个容器
认证	Node 里自建 jose 双 token	**Spring Security** 签发校验，同一套 access/refresh 语义	前端不再持有任何 token 逻辑
数据访问	Prisma	Spring Data JPA + Flyway 管迁移	表结构需重写为 SQL 迁移，schema 语义不变
表归属	单服务	api 拥有业务表，ai 只拥有 `ai_conversations`	需要只读视图与迁移单点
AI 调用	Next 内直接调模型	FastAPI 独立服务，Nginx 直连（不经 Spring）	少一跳，但 AI 服务要自己做限流与鉴权
契约	无显式契约	两份 OpenAPI 生成前端类型	新增契约漂移风险
部署	2C4G 宽松	2C4G **偏紧**，且不能在服务器上构建	需 CI 构建 + 内存预算

1. 目标架构

1.1 拓扑



同域是关键：三个服务共用 `blog.example.com`，Cookie 才能被三者共享（Path=/，HttpOnly），前端 SSR 也能把 `Cookie` 透传给 Spring。不做跨域，避免 CORS + 预检 + Cookie 三方折腾。

1.2 职责边界

服务	只做	明确不做
web（Next.js）	页面渲染、表单、Markdown 渲染、热力图、SSE 消费、SEO	不写业务规则、不连数据库、不签 token
api（Spring Boot）	认证授权、全部业务规则、积分事务、文件上传、迁移（Flyway）	不渲染页面、不调模型
ai（FastAPI）	两段 prompt 组装、模型调用、SSE 输出、AI 限流与成本熔断	不碰业务表、不做用户认证

1.3 为什么 AI 不经过 Spring 转发

- 访客问答是**公网匿名**能力，经过 Spring 也不会获得额外权限校验，只是多一跳。
- SSE 经 Spring 转发需要 WebFlux/`StreamingResponseBody` + 线程占用，2C4G 上不值得。
- 直连后：Nginx 做连接级限流，FastAPI 做业务限流（每日上限、会话轮数），职责清晰。

代价：AI 服务直接暴露公网。缓解——它只有两个端点、无用户数据、限流在 Nginx + 应用双层、`AI_ENABLED` 可一键关停。**内部端点 `/v1/summarize` 用 HMAC 头保护，只允许 Spring 调用**（Nginx 层直接拒绝外网访问该路径）。

---

2. 仓库结构

```
teamblog/
├─ web/                  # Next.js 15 + TS + Tailwind + shadcn
│  ├─ app/
│  ├─ lib/api/           # openapi-typescript 生成的类型 + fetch 封装
│  └─ package.json
├─ api/                  # Spring Boot 3.4 + Java 21 + Maven
│  └─ src/main/java/com/teamblog/
│     ├─ auth/           # SecurityFilterChain、JWT、刷新令牌
│     ├─ user/ project/ learning/ checkin/ points/ comment/ feedback/
admin/
│     └─ common/         # 错误信封、异常处理、审计
│     └─ src/main/resources/db/migration/   # Flyway V1__init.sql ...
├─ ai/                   # FastAPI + Python 3.12 + uv
│  ├─ app/{main,routers,services,prompts}.py
│  └─ pyproject.toml
├─ ops/                  # docker-compose.yml、nginx.conf、backup.sh
└─ docs/                 # 指向 实施方案/ 的说明，或直接放本目录
```

三个服务各自独立构建、独立版本。`ops/` 只放编排，不放代码。

2.1 契约先行（三服务最大的协作风险）

- Spring 用 `springdoc-openapi` 暴露 `/api/v3/api-docs`，**只在本地/CI 生成**，不对公网开放。
- 前端用 `openapi-typescript` 生成 `web/lib/api/schema.d.ts`，用 `openapi-fetch` 做类型安全请求，**手写接口类型视为不合格。**
- FastAPI 自带 `/ai/openapi.json`，前端只消费它的 SSE，不需要生成客户端。
- CI 步骤：启动 api → 拉 openapi.json → 重新生成 → `git diff --exit-code`，**契约变了没同步类型就红。**
- 契约变更流程：改后端 → CI 生成 → PR 里同时带上 `.d.ts` → review 时看 diff 就知道影响了哪些页面。

---

3. 认证与授权（Spring Security）

3.1 语义不变，实现换人

v1.1 定的三条不变：Access 30 分钟、Refresh 7 天、payload 带 `role`、每个写接口后端校验。

项	v1.1（Node）	v1.2（Spring）
签发	`jose`	`NimbusJwtEncoder`（HS256，`JwtSecretKey`）
校验	手写 `verifyJWT`	`spring-boot-starter-oauth2-resource-server` 的 `JwtDecoder`
注入	自己挂 ctx	`SecurityContextHolder` + `@AuthenticationPrincipal`
角色	手写 if	`@PreAuthorize("hasRole('ADMIN')")` + 归属校验
刷新	自建表 + 轮换	同一张 `refresh_tokens` 表，逻辑写在 `RefreshTokenService`
密码	argon2id	`BCryptPasswordEncoder(12)`（Spring 原生；如坚持 argon2 需加 BouncyCastle）

3.2 过滤器链

```
@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.csrf(c ->
c.csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse()))
        .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/api/v1/auth/login", "/api/v1/auth/refresh",
                             "/api/v1/health",
"/api/v1/invites/**").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/v1/users/**",
"/api/v1/projects/**",
```

```
                                "/api/v1/learning-logs/**",
"/api/v1/stats/**",
                                "/api/v1/comments").permitAll()
                .requestMatchers(HttpMethod.POST, "/api/v1/feedback",
"/api/v1/ai/**").permitAll()
                .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class)
            .exceptionHandling(e -> e
                .authenticationEntryPoint((req, res, ex) -> write(res, 401,
"UNAUTHENTICATED"))
                .accessDeniedHandler((req, res, ex) -> write(res, 403,
"FORBIDDEN")));
        return http.build();
    }
```

两条纪律：

1. **URL 级规则只是第一道**，资源归属必须在 service 里再判（`project.ownerId.equals(currentUserId)`），否则成员改 URL 里的 id 就能编辑他人成果。
2. `@PreAuthorize` 加在 controller 方法上，且 **service 不信任 controller 传来的 role**，需要时从 `SecurityContext` 取。

3.3 JWT 过滤器里必须做的三件事

```
// 1) 解 token 2) 查库确认用户未被删、tokenVersion 未变 3) 以库里的 role 构造权限
var user = userRepository.findById(claims.getSubject())
        .filter(u -> u.getDeletedAt() == null)
        .orElseThrow(() -> new InvalidTokenException());
if (!user.getTokenVersion().equals(claims.get("tv", Integer.class))) {
    throw new InvalidTokenException();   // 改密码/改角色/被移除 → 立刻失效
}
var auth = new UsernamePasswordAuthenticationToken(
        user, null, List.of(new SimpleGrantedAuthority("ROLE_" +
user.getRole())));
```

`tokenVersion` 查询是每请求一次主键查询，走连接池，成本可接受；**不要为了省这次查询把 role 完全信任 token**，改角色后旧 token 还能用 30 分钟。

3.4 前端如何带凭证

- 浏览器 → Nginx → Spring：Cookie 自动携带，前端 `fetch` 加 `credentials: 'include'`。
- Next 服务端渲染 → Spring：`cookies().toString()` 透传 `Cookie` 头，**不要**在服务端再登一次；`fetch` 必须 `cache: 'no-store'`，否则会把某个用户的页面缓存给另一个用户（这是最危险的一类 bug）。
- CSRF：Spring 的 `CookieCsrfTokenRepository` 下发可读的 `XSRF-TOKEN`，前端每次写请求带 `X-XSRF-TOKEN` 头。`SameSite=Lax` 是第二道。

3.5 开发期同源

Next 的 `rewrites()` 把 `/api` 和 `/ai` 代理到本地端口，生产由 Nginx 负责，**开发与生产行为一致，且不需要 CORS**：

```
// web/next.config.ts
async rewrites() {
  return [
    { source: '/api/:path*', destination: `${process.env.API_ORIGIN}/api/:path*` },
    { source: '/ai/:path*',  destination: `${process.env.AI_ORIGIN}/ai/:path*` },
  ];
}
```

---

4. 后端模块划分（Spring）

4.1 包结构

```
com.teamblog
├─ auth        # SecurityConfig, JwtFilter, JwtService, RefreshTokenService, AuthController
├─ user        # User, UserRepository, UserService, UserController, AdminUserController
├─ project     # Project, ProjectType/Status, ProjectService（含归属校验）
├─ learning    # LearningLog, ...
├─ checkin     # CheckinService（事务：打卡 + 积分账本 + 连续段判定）
├─ points      # PointEvent, PointsService, LevelCalculator, 对账定时任务
├─ comment     # Comment（二级树组装）
├─ feedback    # Feedback, AdminFeedbackController
├─ admin       # AdminInviteService
└─ common      # ApiResponse, ErrorCode, GlobalExceptionHandler, RequestIdFilter, RateLimiter
```

4.2 约定

- **Controller 只做 DTO ↔ 领域转换**，不含 if 分支业务；`@valid` 做参数校验，Bean Validation 注解与前端 zod 规则一一对应。
- **事务边界在 service**：打卡、积分、成果创建都在 `@Transactional` 内完成，`total_points` 用 `UPDATE ... SET total_points = total_points + ?` 而非先读后写，避免并发丢分。
- **错误统一**：`@RestControllerAdvice` 把异常映射成 `{ok:false,error:{code,message}}`，与 FastAPI 返回格式一致，前端只写一套错误处理。
- **时间**：`-Duser.timezone=Asia/Shanghai` + 代码里一律 `LocalDate.now(ZoneId.of("Asia/Shanghai"))`，打卡用 `LocalDate` 映射 PG `date`，不碰 `Instant`。
- **分页**：cursor 用 `(created_at, id)` 复合条件，`Pageable` 只用于后台表格。

4.3 相对 v1.1 的 4 个具体改动

项	v1.1	v1.2
迁移	`prisma migrate`	**Flyway** `V1__init.sql`，只允许通过迁移改表；JPA `ddl-auto=validate`
主键	`cuid()` 字符串	`uuid`（应用侧生成 UUIDv7，便于排序）；仍不可枚举
枚举	PG enum 类型	`varchar(20)` + `CHECK` 约束；JPA `@Enumerated(EnumType.STRING)`，避免 Hibernate 与 enum 类型打架
积分对账	脚本	`@Scheduled(cron="0 10 3 * * *")`，差异写日志 + 告警

`ddl-auto=validate` 是硬要求：它保证代码里的实体与 Flyway 迁移一致，防止"开发机自动建表、生产迁移缺列"。

---

5. 数据层（两服务共用一库的规矩）

5.1 表归属

表	拥有者	其他服务的权限
`users / refresh_tokens / projects / learning_logs / checkins / point_events / comments / feedbacks / admin_invites`	api	ai 只读 `projects` 的公开字段（走视图）
`ai_conversations`	ai	api 只读（后台审计用）

5.2 给 AI 服务的只读视图

```sql
CREATE VIEW v_projects_public AS
SELECT id, type, title, description, content_md, status
FROM projects WHERE status = 'ACTIVE';

CREATE ROLE ai_reader LOGIN PASSWORD '...';
GRANT SELECT ON v_projects_public TO ai_reader;
GRANT SELECT, INSERT ON ai_conversations TO ai_reader;
```

AI 服务用 `ai_reader` 连接串，物理上不可能改业务数据——比"约定好不改"可靠。

5.3 迁移单点

只有 api 服务跑 Flyway（含 `ai_conversations` 的建表）。AI 服务启动时只做连接健康检查，绝不 `create_all`。这样表结构只有一个真相，回滚也只有一条路径。

---

6. AI 服务（FastAPI）

6.1 端点

方法	路径	鉴权	说明
POST	`/ai/v1/project-chat`	公开 + 限流	`{project_id, conversation_id?, message}` → SSE
POST	`/ai/v1/summarize`	**HMAC 内部头**	`{log_id}` 或 `{content}` → `{summary}`，仅 Spring 可调
GET	`/ai/health`	公开	上游连通性 + 队列深度
GET	`/ai/metrics`	内网	调用量、错误率、token 消耗（Prometheus 文本格式）

6.2 流式骨架

```python
@router.post("/v1/project-chat")
async def chat(req: ChatRequest, request: Request):
    async def gen():
        async for chunk in llm.stream(build_prompt(req)):
            if await request.is_disconnected():      # 用户关页面立刻停，不继续烧 token
                break
            yield f"event: delta\ndata: {json.dumps({'t': chunk})}\n\n"
        yield "event: done\ndata: {}\n\n"
    return StreamingResponse(gen(), media_type="text/event-stream",
                             headers={"X-Accel-Buffering": "no", "Cache-Control": "no-store"})
```

要点：`is_disconnected()` 每轮检查；`event: delta/done/error` 三种事件与前端约定死；错误也要走 SSE（HTTP 已经 200 了，不能改状态码）。

6.3 限流与预算

层	位置	策略
连接级	Nginx `limit_req_zone $binary_remote_addr`	10 r/m，burst 5，超限 429
业务级	FastAPI 内存滑动窗口	10 次/分钟/IP、200 次/天/IP
全局	PG 计数表 `ai_usage_daily`（ai 服务写）	2000 次/天，超限熔断

层	位置	策略
熔断	连续 10 次上游 5xx	打开 5 分钟，直接返回"服务繁忙"

Nginx 先挡一层，应用层再挡一层：Nginx 挡的是暴力刷，应用层挡的是"换 IP 慢速刷"。

6.4 依赖最小化

`fastapi / uvicorn / pydantic-settings / openai / asyncpg / httpx`，不引 ORM、不引 Celery。AI 服务要能在 200MB 内存里跑起来，这是 2C4G 上的生存线。

---

7. 前端（Next.js）定位收缩

- 只做 UI + SSR，**不再有 `app/api/**` 业务路由**；仅保留一个 `/api/health` 供 Nginx 探测。
- 服务端组件调 Spring：`fetch(API_ORIGIN + path, { headers: { cookie }, cache: 'no-store' })`。
- 客户端组件调 `/api/...`（同源），由 Nginx/rewrites 转给 Spring。
- 写操作后 `revalidateTag` / `router.refresh()`，不做全局状态库；只有"当前用户"用 Context。
- AI 对话在浏览器直接连 `/ai/v1/project-chat`，用 `fetch` + `ReadableStream` 消费（POST 不能用 EventSource）。

---

8. 部署编排

8.1 服务与路由

```nginx
# ops/nginx.conf（节选）
limit_req_zone $binary_remote_addr zone=ai:10m rate=10r/m;

upstream web { server web:3000; }
upstream api { server api:8080; }
upstream ai  { server ai:8000;  }

location /api/     { proxy_pass http://api; proxy_read_timeout 60s; }
location /ai/      {
    limit_req zone=ai burst=5 nodelay;
    location /ai/v1/summarize { deny all; }        # 只允许内网调用
    proxy_pass http://ai;
    proxy_buffering off;
    proxy_read_timeout 120s;
    proxy_http_version 1.1;
    add_header X-Accel-Buffering no;
}
location /uploads/ { alias /var/www/uploads/; expires 30d; }
location /         { proxy_pass http://web; }
client_max_body_size 6m;
```

8.2 内存预算（2C4G 的真实约束）

组件	预算	手段
PostgreSQL	~800MB	`shared_buffers=512MB`、`work_mem=8MB`、`max_connections=50`
Spring Boot	~800MB	`-Xmx512m -XX:MaxMetaspaceSize=192m -XX:+UseSerialGC`（小堆用 Serial 比 G1 省）
Next.js	~350MB	standalone + `NODE_OPTIONS=--max-old-space-size=320`
FastAPI	~200MB	uvicorn `--workers 1`，不加载 torch 一类重依赖
Nginx	~30MB	—
合计	**~2.2GB**	剩余 ~1.6GB 给 page cache，够用但无冗余

结论：4G 能跑，但没有余量。 两条硬规矩：

1. **服务器上不构建镜像。**三个镜像的 `docker build`（Maven 下载依赖 + Next 编译 + Python 依赖）在 2C4G 上必然 OOM。用 GitHub Actions 构建并推到镜像仓库，服务器只 `pull`。
2. 配 **2G swap** 兜底，并把 `vm.swappiness` 设 10，避免频繁换页。

如果预算允许，**升到 4C8G 是最划算的一次升级**——三服务同机后，2C 上 Spring 启动 + Next 冷启动会互相抢 CPU。

8.3 启动顺序

`db`（healthy）→ `migrate`（一次性，跑 Flyway）→ `api` → `ai` / `web` 并行 → `nginx`。api 的 `/api/v1/health` 里要包含"Flyway 版本已是最新"，否则会出现 web 起来了但表还没建完的 500。

---

9. 可观测性

- Nginx 生成 `X-Request-Id` 并透传，Spring 与 FastAPI 都把它写进日志与响应头，跨服务排查靠它串。
- 日志统一 JSON：`ts, level, service, request_id, user_id?, msg`。三个服务格式一致，`docker logs | jq` 就能用。
- 关键指标：登录失败率、403 次数、打卡 QPS、AI 首字节延迟（TTFT）、AI 日调用量、积分对账差异。
- AI 的 TTFT 单独打点：流式体验坏掉时，用户感知是"卡"，日志里必须能定位是模型慢还是网络慢。

---

10. 测试

层	内容	工具
api 单测	积分、连续段、等级、归属校验	JUnit 5 + AssertJ

层	内容	工具
api 集成	权限矩阵（member token 打 admin 接口 → 403）、幂等打卡	`@SpringBootTest` + MockMvc + **Testcontainers(PG)**
api 迁移	Flyway 从空库跑到最新 + 种子	Testcontainers，CI 每次跑
ai	prompt 拼装、限流、SSE 事件序列、断连停止	pytest + `httpx.AsyncClient`
契约	OpenAPI 生成 + 类型 diff	CI 脚本
e2e	访客问项目出字、成员打卡看到积分、管理员下架成果	Playwright

禁止用 H2 替代 PG 跑集成测试——`date`、`jsonb`、唯一索引冲突语义都不一样，测过等于没测。

---

11. 排期调整（12 个工作日）

三栈并行，契约先行是前提。D1 必须产出 OpenAPI 草案（哪怕只有 auth + checkin），否则前端只能等。

天	api（Java）	web（Next）	ai（Python）	测试/运营
D1	骨架 + Flyway V1 + 契约草案	骨架 + 类型生成流水线	骨架 + 模型连通性验证	评审 + 用例表
D2–3	认证/刷新/过滤器链 + 权限矩阵	登录、邀请、导航、页面壳	prompt 初版 + 离线评测 20 条	造数脚本
D4–5	用户、成果、学习内容 + 上传	主页、个人页、成果流/详情	—	录真实内容
D6	打卡 + 积分 + 热力图查询	热力图、打卡交互	—	积分单测复核
D7 门禁	**登录/内容/打卡/部署跑通，AI 才接前端**			
D7–8	评论、留言、后台、对账任务	评论、留言、后台四 tab	项目问答 SSE + 限流	权限矩阵用例
D9	部署编排、健康检查、备份	移动端、SEO、错误态	摘要接口 + 熔断	恢复演练
D10	联调修复	联调修复	压测（并发 10）	回归
D11	上线检查 6 条	视觉走查	成本核对	验收
D12	缓冲：跨服务问题、文档			

比 v1.1 多 2 天，主要花在：契约生成流水线（0.5 天）、三服务部署与联调（1 天）、Java 侧认证与权限测试（0.5 天）。

---

12. 新增风险

风险	概率	影响	缓解
三语言栈，四人团队上下文切换成本	高	中	一人一栈主责（Java/TS/Python/测试），交叉 review 只读不写
契约漂移：后端改了字段前端不知道	高	中	OpenAPI 生成 + CI diff 卡红
服务器上构建 OOM	高	高	CI 构建推镜像，服务器只 pull；2G swap 兜底
2C 上 Spring 启动与 Next 冷启动抢 CPU	中	中	错峰启动 + `-Xmx512m` + 就绪探针
AI 服务公网暴露被刷	中	中	Nginx + 应用双层限流、日上限、`AI_ENABLED` 开关
两服务共库导致越权写	低	高	只读账号 + 视图，物理隔离
Spring 的 CSRF 与前端 fetch 不匹配	中	低	统一 `X-XSRF-TOKEN`，e2e 覆盖一次写请求
ISO 时区在 JVM 上被忽略	低	高	`-Duser.timezone` + 显式 `ZoneId`，跨时区单测

---

13. 待确认（v1.2 新增）

#	问题	默认值
Q11	Java 版本 / 构建工具	Java 21 LTS + Maven（团队熟悉则换 Gradle）
Q12	是否引入 Redis	一期不引；限流走内存 + PG 计数，扩容再说
Q13	Spring 数据访问	JPA + 少量 native query（热力图、对账），不引 MyBatis
Q14	前端是否保留 SSR	保留（访客页面要 SEO），但只 SSR 公开页；`/me`、`/admin` 可退化为客户端渲染
Q15	图片存储	一期本地卷 + Nginx 直出，二期对象存储
Q16	AI 服务是否需要独立数据库	不需要，共库 + 视图隔离；量级上来再拆

附录：与 v1.1 的章节对应

v1.1 章节	v1.2 处理
§1 架构总览	由本文 §1 取代
§2 认证与授权（含 guard 代码）	由本文 §3 取代；**权限矩阵 §2.4 继续有效**
§3 数据模型	字段不变，落地方式见本文 §4.3、§5
§3.4 迁移与种子	由 Flyway 方案取代（本文 §5.3），种子改为 `R__seed.sql` 或 test profile
§4 打卡·积分·等级	**完全继续有效**（规则与服务侧实现无关）
§5 接口清单	路径前缀从 `/api/v1` 保留；AI 端点改为 `/ai/v1/*`；错误格式不变
§6 页面与前端	继续有效，但去掉"Next 内 API 路由"的隐含假设
§7 AI 方案	prompt 与降级继续有效；实现改为 FastAPI（本文 §6）
§9 模板选型	已不适用（前端只做 UI，不存在"全栈模板"问题）
§10 部署	由本文 §8 取代
§11 测试	由本文 §10 取代
§12 排期	由本文 §11 取代（10 天 → 12 天）